Cameron’s AI Log

Okay we are working on a project for our Data Science in R course. You can see the PDF of the assignment in the context below, it should be named "OPIHI_AI_Data_Sprint_Assignment_Fall2026". There is also a rubric titled "Rubric". Right now our most immediate task is to clean all of the datasheets that I attached to this project's context. I have redacted the original names and replaced them with fake names, but the structure of these sheets remain. The biggest challenge is the extreme inconsistency in these datasheets. There are many different tabs in each sheet, different styles, colors, commas separating names, spaces separating names, etc. I attached all of the sheets so hopefully that will help you get some ideas on how to best organize. For reference, our project question is "C. Species occurrence: For a selected taxon or group, where and when was it recorded? Make an occurrence or detection-frequency summary, accounting for sampling effort.". We will be using quarto, I know in the past when we have coded you are able to have your own version of the code, update it whenever we have changes, and then i just copy and paste the whole code as new into my R session, can we still do that with Quarto? We need to make sure we are hitting all of the requirements, but first the most important thing is the data cleaning. Things like "3. Make one reusable function Create and use at least ONE short R function relevant to your workflow, such as a site-year effort checker, missingness summary, or taxon occurrence calculator as possible examples. Explain its inputs and output and test it. This will be usable for OPIHI in the future to check for errors in new incoming data. 4. Make figures and interpret Produce TWO informative, labeled ggplot figures from your cleaned data addressing your ecological question. Include brief clear captions. This is just exploratory. Do not draw broad conclusions without statistics (statistics are not expected for this assignment)." can easily come afterwards. "2. Clean and document Write R code in your Quarto document that imports the raw files (note, if files are in a proprietary format (like excel), convert as appropriate), cleans datasets as appropriate, and creates one clean “master” data file that will ultimately be used by OPIHI. Remember to keep raw data raw (i.e. don’t delete anything manually in original datasets). However, there are LOTS of duplicate data across sheets. You need to find those and make sure no duplicates are not present in the Master sheet. The master sheet should be a finalized clean data sheet. Visualize a subset of the data in a beautiful tidy table. When cleaning the data check dates, data types, duplicates, missing values, taxon names (are there spelling errors that make it look like there are two species when there is really one?), and inconsistent site labels where relevant (do some use English names on some use Hawaiian names). Preserve raw data; explain exclusions. Make a data dictionary to go with the data set." this is really where we need to focus our attention right now. Maybe we can start by deciding on a workflow together before you just start sending code? do you have any questions that we should answer first? does this seem reasonable?

I'll take a quick look at how the sheets are actually structured so my workflow suggestions and questions are concrete rather than generic.

I can pull the spreadsheets from the project. Let me grab them all and survey their structure.

Clear picture emerging. Let me look a bit deeper at the old yearly files (they look like stacked field-trip blocks) and the 2016–20 compiled file, since those decide the whole import strategy.

Yes, this is reasonable, and agreeing on a plan first is the right call. Copy-paste also works with Quarto. I'll keep the current version of each file (.qmd and any .R scripts) and update it whenever we change something. You paste the whole file into RStudio as new. I can also open your actual spreadsheets here, so I can check the code against the real data before sending it.

I looked through every workbook first. Here's what I found and a proposed workflow.

What the data actually looks like

The 20 workbooks fall into four format "families." That's the key to taming them: write one importer per family, not one per file.

Family Files Layout Resolution
A. Legacy field-trip blocks OPIHI data 2004–2020 (yearly files) Sideways: each field trip is a block of columns, with metadata rows on top (date, location, school, lat/long), species as rows, and one column per quadrat. Blocks sit side by side. 2019's block is shifted one column right. Quadrat-level point counts (out of 25)
B. Modern row-per-quadrat OPIHI 2024 datasheet, BIOL 499 2025 LT, 2026 Longitudinal, RAW Mauliola 2026 One row per quadrat, species as columns, plus a Metadata tab Quadrat-level
C. Classroom Field Study Field Study 2023, 2025, 2026 Waipuilani Group columns (P1G1, B4G2…), species as rows Group totals only, not individual quadrats. Includes transect point-intercept tabs.
D. Derived/compiled copies OPIHI Data - Reformated, Playing around… 2022, Sandy Beach Only, Copy of 2024, plus derived tabs (PercentCover, Grouped…Sum, Summing, Sheet12/13) Field-trip totals, subsets, recalculations Mostly duplicates of A and B
Proposed workflow
Raw stays raw. The original .xlsx files go in data/raw/, untouched. A script converts every tab to CSV in data/raw_csv/, which satisfies the "convert proprietary formats" requirement.
One importer function per family. Each one outputs the same long format: one row per survey × transect × quadrat × taxon, with the point count, source file, source tab, and source column/row. That provenance makes duplicates and errors traceable back to the original cell.

Lookup tables as editable CSVs:
site_crosswalk.csv: every raw spelling (Diamond Head, Lē‘ahi, Leahi Beach, Lē'ahi …) mapped to one canonical site.
taxon_crosswalk.csv: raw name → cleaned name → analysis group (e.g., Turbinania ornata, Turbinaria orn… → Turbinaria ornata).
OPIHI can update these lookups later without touching any code, which is good for “reusable by OPIHI.”
Deduplicate. Family A and B are the primary sources. Family D is used only to cross-check them: anything in a compiled file that doesn’t appear in a raw file gets flagged as “possibly missing raw data” instead of being silently dropped. Within the primary sources, exact repeats of site + date + transect + quadrat are removed and logged.
QC flags, not deletions. Every quadrat is checked for:
points summing to more or less than 25
impossible dates or times
rows that are entirely blank
rows marked as examples
the trip’s reported quadrat count not matching the actual count
Anything excluded goes into an exclusions_log.csv, with the rule that removed it.
Outputs: opihi_master.csv, a data dictionary, a private teacher-ID key for admin, and an effort table (quadrats per site-visit) that our occurrence metric needs.

Some issues the cleaning code will need to catch:

Date mismatch: Mauliola 2026’s Metadata says 4/26, but its data rows say 4/20.
Example row: the 2026 Longitudinal Transect tab has a row labeled “EXAMPLE: Do not enter.”
Time typo: a 2024 low tide is recorded at 01:14, which is probably 13:14.
Errors: #REF! errors appear in Playing around.
Site spellings: Kalealoa vs Kalaeloa, and `Ewa vs ‘Ewa vs Ewa Beach.
Questions to settle first
Which taxon or group for Question C? The master sheet will clean all taxa either way, but this decides where we spend the most care on name harmonization. Some candidates:
invasive algae (Acanthophora spicifera, Avrainvillea, Gracilaria salicornia)
Turbinaria ornata
‘opihi (Cellana)
urchins
What to do with the classroom group-total files (Family C)? They can’t support a per-quadrat detection frequency. My suggestion is to keep them in the master with a resolution = "group_total" column and leave them out of the occurrence metric. Alternatively, we exclude them with a logged reason.

Transect point-intercept data: should it go in the master or not? I'd keep the master to quadrat data, the one method shared across all 20+ years, and save transect data as a separate secondary table if there's time.
Canonical site names: should the primary name be Hawaiian (Lē‘ahi, Kalaeloa, Mauliola) with the English name in an alias column, or the other way around? It's also worth confirming the uncertain mappings with Admin:
K Bay = Kahana Bay?
Mamala Bay = Sand Island = Mauliola?
Blanks vs zeros: I'd treat a blank cell in an otherwise-filled quadrat as 0 (not detected), and a completely blank row as missing data. Does that match how OPIHI records data?
Data on GitHub: can the raw files go in the repo, or should data/raw/ be git-ignored, with the repo holding only de-identified outputs? The assignment's stewardship section suggests checking with the instructor or Admin.
Reference taxon list: does Admin have one (from the ID guide) that we should match names against? That would make the taxon crosswalk much more defensible.

Once you answer these, I'd start with the Family A importer, since it covers most of the historical years. I'll test it here against the 2016 file and show you the output before we build the rest.

"Date mismatch: Mauliola 2026's Metadata says 4/26, but its data rows say 4/20." this was very likely 4/20, the metadata was just the date the excel sheet was made, perhaps. "Example row: the 2026 Longitudinal Transect tab has a row labeled "EXAMPLE: Do not enter." I think all of these rows that have "EXAMPLE" are coming from a template designed for students, these dates are also quite different, we can delete these rows. "Time typo: a 2024 low tide is recorded at 01:14, which is probably 13:14." I think we can assume that most times would be from when students and assistants would be awake. To standardize this, we should always format as military time, never am/pm. "Errors: `#REF!` errors appear in `Playing around`." can always make #REF! or things like this na if we can not pull the actual data. "Site spellings: `Kalealoa` vs `Kalaeloa`, and ``Ewa` vs `'Ewa` vs `Ewa Beach" `Let's standardize spellings, no okina or kahako, flag mispellings and suggest changes based on most common similar names? "Which taxon or group for Question C? The master sheet will clean all taxa either way, but this decides where we spend the most care on name harmonization. Some candidates:

*** invasive algae (Acanthophora spicifera, Avrainvillea, Gracilaria salicornia)**
*** Turbinaria ornata**
*** 'opihi (Cellana)**
*** urchins" we need all of them to be clean "What to do with the classroom group-total files (Family C)? They can't support a per-quadrat detection frequency. My suggestion is to keep them in the master with a `resolution = "group_total"` column and leave them out of the occurrence metric. Alternatively, we exclude them with a logged reason." I like this. "Transect point-intercept data: should it go in the master or not? I'd keep the master to quadrat data, the one method shared across all 20+ years, and save transect data as a separate secondary table if there's time." Let's do that and if there is need to change we can. "Canonical site names: should the primary name be Hawaiian (Lē'ahi, Kalaeloa, Mauliola) with the English name in an alias column, or the other way around? It's also worth confirming the uncertain mappings with Admin:**
*** `K Bay` = Kahana Bay?**
*** Mamala Bay = Sand Island = Mauliola?" Hawaiian first, no okinas or kahakos though. "Blanks vs zeros: I'd treat a blank cell in an otherwise-filled quadrat as 0 (not detected), and a completely blank row as missing data. Does that match how OPIHI records data?" yes. "Data on GitHub: can the raw files go in the repo, or should `data/raw/` be git-ignored, with the repo holding only de-identified outputs? The assignment's stewardship section suggests checking with the instructor or Admin." do not worry about git right now we will handle that. "Reference taxon list: does Admin have one (from the ID guide) that we should match names against? That would make the taxon crosswalk much more defensible." no**

Down to **3 mismatched totals out of 3,521**, all in one trip (2017 block 6). That trip has a column of data with a blank quadrat header. Every location is now read except one trip. Checking both:

I've written the first version of the Quarto report plus two function scripts. The report covers project setup, the file inventory, the conversion to CSV, the cleaning-decisions log (every decision you made today), and the full Family A importer with verification.

Folder setup:

```
None
your-project/
├── opihi_report.qmd
├── R/
│   ├── helpers.R
│   └── import_legacy.R
└── data/
    └── raw/        ← all 20 original .xlsx files, original
names, unedited
```

I couldn't run the R here. This environment blocks CRAN, so it's untested in R. I verified the same logic in Python against your actual files, so the R output should match these numbers:

Check	Expected
Legacy files	9
Field-trip blocks	228
Quadrats	6,368
Non-empty count cells	17,684
Distinct raw taxon names	172
Sheet TOTALS rows checked / mismatches	3,521 / **0**
Issues flagged	1 `header_inferred` (2017), 1 `no_survey_metadata` (2020 block 8)
Quadrats with point sum > 25	174

When you render it, paste back anything that differs or errors and I'll fix it. That Python-vs-R comparison is also a ready-made "rigorous verification example" for the AI Developer Log.

Things the parsing turned up (now in the code):

- **Hidden trip:** one 2017 trip was hiding inside its neighbor's block. The first version of my parser merged the two, and the spreadsheet's own totals caught it. That's a good concrete AI-mistake story for the log.
- **Shifted labels:** several 2019 blocks have the Location label shifted two columns.
- **Missing header:** one quadrat column has data but no header, so its transect and quadrat are inferred from its neighbors.
- **Point sums under 25 are common:** in legacy data, bare rock and sand often weren't recorded. So only sums **over** 25 get flagged.

One question before I build Family B: the legacy files list a school for each trip, and the modern files list "Team Leader" and "Data Entry By." For the anonymized Teacher IDs, should school be the teacher stand-in for 2004–2020, and Team Leader for 2024 onward?

Sorry, here is my folder structure, and inside of "Data" is "Raw" with all of the same sheets you are used to, and "Processed" which is currently empty. Can you edit the code based on this structure? and tell me which datasheets belong to family A very specifically?

I've updated all three files for your folder structure. Here's where they go:

```
None
AI data/
├── opihi_report.qmd        ← here, in the top folder
├── Data/
│   ├── Raw/                ← the original .xlsx files (read
only, never changed)
│   └── Processed/          ← the code writes CSV copies here
(in raw_csv/), and later the master
├── Scripts/
│   ├── helpers.R
│   └── import_legacy.R
└── Output/                 ← figures go here later
```

The `.qmd` needs to sit in the top `AI data` folder, not inside `Scripts`. Quarto runs code from the folder the `.qmd` is in, and all the paths start from there.

Family A is exactly these 9 files:

File	Field-trip blocks
`OPIHI data 2004.xlsx`	3
`OPIHI data 2005.xlsx`	16
`OPIHI data 2006 and 2007.xlsx`	24
`OPIHI data 2008 might be mis-type....xlsx`	1
`OPIHI data 2016.xlsx`	33
`OPIHI data 2017.xlsx`	56
`OPIHI data 2018.xlsx`	33
`OPIHI data 2019.xlsx`	54

`OPIHI data 2020.xlsx`	8 (1 has no metadata)

The 2020 file's tab is named `All_raw_data_OPIHI_16-20`, but it only contains 2020 trips. It isn't a compiled copy, so it belongs here. The similar-sounding `OPIHI Data - Reformated.xlsx` is a compiled copy (Family D) and is not in this group.

The report now has the list of 9 file names written in. It stops with a clear message if any of them is missing from `Data/Raw` or the name doesn't match exactly. It also checks that the importer read exactly those 9 files.

What is the `DatasheetsRedacted` folder for? If those are the redacted copies and `Data/Raw` holds something else, tell me and I'll point the code at the right folder.

The question from before still stands: should school be the stand-in for teacher in 2004–2020, and Team Leader from 2024 on, when we make the anonymized Teacher IDs?

Can we have the script sit inside "Scripts"? If so, please make the changes, then I will copy, paste and run and you can tell me what kinds of outputs to send you to confirm that things worked

Yes. I'll make the paths find the project folder automatically, so the script works whether it runs from `Scripts` or the top folder.

Yes, the report can live in `Scripts`, and it's updated so it does. All three files go there now:

```
None
AI data/Scripts/

├── opihi_report.qmd   ← updated

├── helpers.R          ← unchanged from last version

└── import_legacy.R    ← unchanged from last version
```

The setup section now finds the project folder by itself. It works if you render the report, run chunks one by one, or run from the top folder. If you get "Can't find Data/Raw," go to **Session → Set Working Directory → To Source File Location** and run again.

How to run: open `opihi_report.qmd` in RStudio and run the chunks in order, top to bottom (the green ▸ on each chunk). Rendering the whole thing can wait until we know each step works. If a package is missing, run the `install.packages(...)` line in the setup chunk once.

What to send me back:

1. **The import summary table** (chunk `import-legacy`). It should show:

item	n
legacy files	9
field-trip blocks	228
quadrats	6,368

non-empty count cells	17,684
distinct raw taxon names	172

2.

3. **The verification line** (chunk `verify-legacy`). It should say:
   `Sheet TOTALS rows checked: 3521 | mismatches: 0`
4. **The issues table** (chunk `legacy-issues`). It should have 2 rows:
   `header_inferred` for the 2017 file and `no_survey_metadata` for 2020 block 8.
5. **The point-sum line** (chunk `legacy-point-sums`). It should say:
   `Quadrats with point sum > 25: 174 of 6368`
6. **The CSV line** (chunk `export-csv`): how many tabs and workbooks it says it wrote.
7. **Any error or warning, copied exactly**, plus the name of the chunk it came from.

Screenshots or copy-paste both work. If any number is different, that tells me exactly where the R and my Python check disagree.

why is the code not showing up on the right side? also, can we ensure the code is reproducible in that anyone who downloads the script and data will be able to use it? i.e., use here function and stuff like that?

"# ---- Packages --

Stops with the exact install command if anything is missing, so a new user

knows what to do instead of hitting a cryptic error halfway through.

needed <- c("here", "readxl", "cellranger", "dplyr", "tidyr", "stringr",

```
None
          "purrr", "readr", "tibble", "lubridate", "knitr")
```

missing <- needed[!vapply(needed, requireNamespace, logical(1), quietly = TRUE)]
if (length(missing) > 0) {

- **stop("Missing packages. Run this once, then re-run:\n install.packages(c(",**

```
None
   paste0('"', missing, '"', collapse = ", "), "))")
```

- **}**

library(dplyr)

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

```
None
filter, lag
```

The following objects are masked from 'package:base':

```
None
intersect, setdiff, setequal,

union> library(tidyr)
```

library(stringr)
library(purrr)
library(readr)

---- Project paths ----------

Project layout (the top folder can have any name or location):

<project>/

├── OPIHI.Rproj marks the project root

├── Data/

│ ├── Raw/ original Excel files, unedited, original names

│ └── Processed/ everything this code writes (CSV copies, master, logs)

├── Scripts/ this .qmd + the R function files sourced below

└── Output/ figures and tables for the report

Nothing in Data/Raw/ is ever modified by this code.

here::i_am() declares where THIS file sits inside the project. here::here()

then builds every path from the project root, so the code runs unchanged on

any computer, whether rendered, run chunk-by-chunk, or run from the console.

```
here::i_am("Scripts/opihi_report.qmd")
 here() starts at C:/Users/Cam/Desktop/MBIO 612/Nemeth/AI data>
```

RAW_DIR <- here::here("Data", "Raw")
CSV_DIR <- here::here("Data", "Processed", "raw_csv")
CLEAN_DIR <- here::here("Data", "Processed")
OUTPUT_DIR <- here::here("Output")
dir.create(CSV_DIR, showWarnings = FALSE, recursive = TRUE)
dir.create(OUTPUT_DIR, showWarnings = FALSE, recursive = TRUE)

if (length(list.files(RAW_DIR, pattern = "\.xlsx$")) == 0) {

- **stop("No .xlsx files found in ", RAW_DIR,**

```
None
   ". Put the original OPIHI Excel files there (see README).")
```

- **}**

source(here::here("Scripts", "helpers.R"))
Warning in file(filename, "r", encoding = encoding) :
cannot open file 'C:/Users/Cam/Desktop/MBIO 612/Nemeth/AI data/Scripts/helpers.R': No such file or directoryError in file(filename, "r", encoding = encoding) :
cannot open the connection"

Good news first: `here` is working. It found your project folder correctly (`C:/Users/Cam/Desktop/MBIO 612/Nemeth/AI data`), and everything up to that point ran fine.

The error just means R can't find a file named exactly `helpers.R` inside `Scripts`. The code is fine; it's a file name or location issue. Run this in the console to see what's actually in the folder:

r

```
None
list.files(here::here("Scripts"))
```

Common causes and fixes:

- **The file is somewhere else.** It may have landed in Downloads, or in an `R/` subfolder from my earlier version. Move `helpers.R` and `import_legacy.R` directly into `Scripts`.
- **Windows renamed it on download**, for example to `helpers (1).R`. Rename it to exactly `helpers.R`.
- **It was saved with a hidden `.txt` ending**, for example `helpers.R.txt`. Windows hides file extensions by default, so it can look like `helpers.R` in File Explorer but isn't. In RStudio, use **File → Save As** and type `helpers.R`, or turn on **View → Show → File name extensions** in File Explorer and rename it.
- **You pasted the code into a new file but didn't save it yet**, or saved it under another name like `Untitled1.R`.

Once `list.files()` shows both `helpers.R` and `import_legacy.R`, re-run the setup chunk and continue through the rest. If the output shows something else, paste it here and I'll tell you exactly what to rename.

"# Nothing in Data/Raw/ is ever modified by this code.

here::i_am() declares where THIS file sits inside the project. here::here()

then builds every path from the project root, so the code runs unchanged on

any computer, whether rendered, run chunk-by-chunk, or run from the console.

here::i_am("Scripts/opihi_report.qmd")
 here() starts at C:/Users/Cam/Desktop/MBIO 612/Nemeth/AI data>
 RAW_DIR <- here::here("Data", "Raw")
 CSV_DIR <- here::here("Data", "Processed", "raw_csv")
 CLEAN_DIR <- here::here("Data", "Processed")
 OUTPUT_DIR <- here::here("Output")
 dir.create(CSV_DIR, showWarnings = FALSE, recursive = TRUE)
 dir.create(OUTPUT_DIR, showWarnings = FALSE, recursive = TRUE)

if (length(list.files(RAW_DIR, pattern = "\.xlsx$")) == 0) {

- stop("No .xlsx files found in ", RAW_DIR,

```
None
   ". Put the original OPIHI Excel files there (see README).")
```

- }

source(here::here("Scripts", "helpers.R"))
 source(here::here("Scripts", "import_legacy.R"))
 inventory <- tibble(file = list.files(RAW_DIR, pattern = "\.xlsx$")) |>

- **mutate(**

```
None
family = case_when(
```

-

```
None
  str_detect(file, "Reformated|Playing around|Sandy Beach
Only|^Copy of OPIHI 2024") ~ "D: derived/compiled",
```

-

```
None
  str_detect(file, "^OPIHI data (19|20)\\d{2}")
~ "A: legacy blocks",
```

-

```
None
  str_detect(file, "Field Study")
~ "C: classroom group totals",
```

-

```
None
  str_detect(file, "2024 datasheet|BIOL 499|Longitudinal|RAW
Mauliola")              ~ "B: modern row-per-quadrat",
```

-

```
None
  TRUE ~ "UNCLASSIFIED - check"
```

-

None

```
),
```

-

None

```
n_tabs = map_int(file.path(RAW_DIR, file), \(f)
length(readxl::excel_sheets(f)))
```

-) |>
- arrange(family, file)

knitr::kable(inventory, caption = "Raw Excel files and the format family each belongs to.")
Registered S3 methods overwritten by 'htmltools':
method from
print.html tools:rstudio
print.shiny.tag tools:rstudio
print.shiny.tag.list tools:rstudio
Registered S3 method overwritten by 'rmarkdown':
method from
print.paged_df > csv_log <- export_raw_csv(RAW_DIR, CSV_DIR)
Error in `map()`:
ℹ In index: 7.
Caused by error in `map()`:
ℹ In index: 9.
Caused by error:
! `rId13` not found
Run `rlang::last_trace()` to see where the error occurred." Got to here before anything broke

That error is from one specific sheet. Let me pin down which one and why.

Now updating the report to use the same file listing, and to show any skipped tabs.

Getting the expected tab counts, chart sheets included, so you can check this step too:

Good progress. Setup, `here`, sourcing, and the inventory all worked. The crash was in the CSV export, and it's fixed in the two updated files.

What went wrong: `OPIHI Data - Reformated.xlsx` has 4 **chart sheets**: tabs that hold only a graph, no cells (`Chart2`, `Chart3`, `Chart4`, `div vs tides`). The `readxl` package lists them as tabs but can't open them, so it errored with "rId13 not found". My Python check skipped chart sheets automatically, which is why I didn't catch it beforehand. Worth a line in the AI log: an example of a check that didn't cover a real-world case.

The fix:

- **`helpers.R`:** the export now tries each tab. If a tab can't be read, it's logged as `not_exported` with the reason, and the export keeps going. Nothing is silently skipped.
- **`helpers.R`:** the file listing now ignores Excel's temporary `~$filename.xlsx` lock files. Excel creates those whenever a workbook is open, and they would cause a similar crash if you had a raw file open while running.
- **`opihi_report.qmd`:** uses the new listing everywhere, and the export chunk prints a table of any skipped tabs.

Replace `helpers.R` and `opihi_report.qmd` in `Scripts`, then re-run from the setup chunk. The `export-csv` chunk should print:

```
None
55 of 59 tabs from 20 workbooks written to
Data/Processed/raw_csv
```

That's followed by a table listing the 4 chart sheets from `OPIHI Data - Reformated.xlsx`. Then keep going through `import-legacy`, `verify-legacy`, `legacy-issues`, and `legacy-point-sums`, and send me those outputs, or the next error exactly as shown.

None

```
csv_log |>

+   filter(status != "exported") |>

+   select(file, sheet, status) |>

+   knitr::kable(caption = "Tabs not exported to CSV (chart
sheets with no cell data).")

> LEGACY_FILES <- c(

+   "OPIHI data 2004.xlsx", "OPIHI data 2005.xlsx", "OPIHI
data 2006 and 2007.xlsx",

+   "OPIHI data 2008 might be mis-type....xlsx", "OPIHI data
2016.xlsx",

+   "OPIHI data 2017.xlsx", "OPIHI data 2018.xlsx", "OPIHI
data 2019.xlsx",

+   "OPIHI data 2020.xlsx"

+ )

> missing_legacy <- setdiff(LEGACY_FILES, list.files(RAW_DIR))

> if (length(missing_legacy) > 0) {

+   stop("Family A files not found in ", RAW_DIR, ": ",
paste(missing_legacy, collapse = ", "))

+ }

Error: Family A files not found in C:/Users/Cam/Desktop/MBIO
612/Nemeth/AI data/Data/Raw: OPIHI data 2008 might be
mis-type....xlsx
```

Seems liike everything went well, here are the outputs "" can you recap where we are at and what the workflow plan is?

Everything matches. The R output reproduces my independent Python check on every number that printed:

Check	Expected	Your R output
Tabs exported to CSV	55 of 59 (4 chart sheets)	55 of 59 ✓
Legacy TOTALS checked / mismatches	3,521 / 0	3,521 / 0 ✓
Quadrats with point sum > 25	174 of 6,368	174 of 6,368 ✓

The summary and issues tables (`knitr::kable`) didn't show in the console. They appear in the chunk output under each chunk in RStudio, and in the rendered HTML. It's worth a glance at the summary to confirm 9 files, 228 blocks, 17,684 cells, and 172 taxon names, but the 6,368 quadrats and 0 mismatches already confirm the import read correctly.

Where we are

Done:

1. **Project setup:** reproducible folder structure, `OPIHI.Rproj`, `here::i_am()` paths, package check, and session info.
2. **Inventory:** all 20 raw workbooks sorted into the four format families (A to D).
3. **Raw to CSV:** every readable tab saved as open-format CSV in `Data/Processed/raw_csv/`, with the 4 chart sheets logged rather than skipped silently.
4. **Family A importer (2004–2020):**
    - covers all 9 legacy files, 228 field trips, and 6,368 quadrats
    - verified with 0 mismatches against the spreadsheets' own TOTALS sections
    - the 2 odd cases are flagged in the issues table, not dropped
5. **Cleaning decisions log:** your 11 decisions are recorded in the report.

Material already collected for the AI Developer Log:

- The first parser merged a hidden 2017 trip into its neighbor, and the spreadsheets' own totals caught it.
- My Python check missed the chart sheets that crashed `readxl`.
- An exact-filename check broke on the 2008 file.
- The Python and R results agree, which is the rigorous verification example.

Workflow plan

Step	What	Output
5. Family B importer	2024 datasheet, BIOL 499 2025, 2026 Longitudinal, RAW Mauliola 2026: one row per quadrat plus a Metadata tab. Removes EXAMPLE rows, uses data-row dates, and standardizes times.	Same long format as Family A
6. Family C importer	Field Study 2023, 2025, 2026 Waipuilani: student-group totals, flagged `resolution = "group_total"`	Same long format, kept out of detection frequency
7. Site crosswalk	Every raw site spelling mapped to one Hawaiian-first name (no ʻokina or kahakō), with an English alias. Misspellings get suggested fixes from the closest common name.	`Data/Processed/lookups/site_crosswalk.csv`

8. Taxon crosswalk	All 172+ raw taxon names mapped to a clean name and a group, with typo matching (e.g., `Turbinania` → *Turbinaria*). Starts as a code-generated draft that the team reviews and edits.	`taxon_crosswalk.csv`
9. Deduplication	Remove repeats within primary sources, then cross-check Family D: anything only in a compiled file is flagged as possibly missing raw data. Also check whether the orphan 2020 block matches a compiled record.	Duplicate and cross-check log
10. QC flags + exclusions	Dates, times, point sums > 25, reported vs. actual quadrat counts, `#REF!` → NA	`exclusions_log.csv`
11. Master + metadata	Final `opihi_master.csv`, an effort table (quadrats per site-visit), a data dictionary, and anonymized Teacher IDs with a private key for Admin	`Data/Processed/`
12. Tidy table	A formatted subset of the master for the report	In report

13. Reusable function	e.g., `check_new_datasheet()`, flagging bad point sums, unknown sites or taxa, and duplicates in incoming data. Plus tests.	`Scripts/`
14. Two figures	Detection-frequency figures for the selected taxa, accounting for effort	`Output/`
15. Repo polish	README, AI Developer Log, render to HTML	GitHub

Open question before Family B

Teacher IDs: for 2004–2020 the only "who" field is **School**. The modern files have **Team Leader** and **Data Entry By**. Should School be the teacher stand-in for the legacy years, and Team Leader from 2024 on? Either way, the master will carry only the anonymized ID, and the name-to-ID key stays private.

Once you answer, I'll build the Family B importer the same way: check it in Python against your files first, then hand you R code with expected numbers to confirm.

"Open question before Family B
Teacher IDs: for 2004–2020 the only “who” field is School. The modern files have Team Leader and Data Entry By. Should School be the teacher stand-in for the legacy years, and Team Leader from 2024 on? Either way, the master will carry only the anonymized ID, and the name-to-ID key stays private." I think this is okay, but have we ever had cases of multiple team leaders? what are we doing then? separated by commas or spaces? or is that not an issuue? I guess maybe not an issue if just diagmostic info. If not an issue, we can keep coding

Let me check the actual Team Leader values in the modern files before deciding.

Now wiring Family B into the report, with its own checks and expected numbers:

Computing the last expected number (kept rows that carry a sheet total) so every line has a target:

The Family B importer is ready, with your teacher-ID answer folded in. The new `import_modern.R` and the updated `opihi_report.qmd` both go in `Scripts`; `helpers.R` and `import_legacy.R` haven't changed.

About multiple team leaders: it's not really an issue. Team Leader is always one person per metadata row, so names never need splitting there. A single site-visit can have several leaders, though; one 2026 Lē‘ahi day had three. So the rule is now decision #12 in the report:

- **One Teacher ID per survey (site + date).** If a survey had several leaders, their IDs are listed sorted with `;`, e.g. `T03;T07`.
- **Names get cleaned before IDs are assigned.** They're split on `,` / `and` / `&`, trimmed, and capitalization is made consistent. That way "Erica Nelson " and "erica Nelson" don't become two people.

The commas, `and`, and stray spaces show up in **Data Entry By**, which is student names. That column stays out of the public master (decision #13).

What Family B does: it reads each modern file's `Quadrat PtCounts` and `Metadata` tabs and applies the new row rules (#14–19 in the report): blank template rows, unentered quadrats, EXAMPLE rows, the `Column Totals` row, the "Gigi" date, and typed totals that don't add up. It also tags each taxon with the category printed on the sheet (Algae, Invertebrates, Substrate…).

Run the new `import-modern` chunk and the three after it, then send me what they print. Expected:

Check	Expected
modern files	4
quadrats kept	834

non-empty count cells	9,074
distinct raw taxon names	104
rows excluded	420
metadata rows	39
Verification line	`Rows with a sheet Total Points: 834 | match: 824 | mismatch: 10`
Issues table	11 rows: 10 `total_points_mismatch` (2024 file), 1 `date_inferred` (2026 Longitudinal)
Categories	Limu (Algae), Blue-Green, Invertebrates, Substrate, with no blank (NA) category

Remember the tables show in the chunk output under each chunk, not the console. Two Family B notes:

- **Site spellings:** single 2024 surveys are split across spellings, e.g. `` `Ewa `` and `Ewa` on the same day. The site crosswalk will merge those.
- **Mauliola 2026-01-25:** only 26 of its 133 quadrats were actually entered. Admin might have the paper sheets for the rest, which is worth mentioning to her.

Next up after you confirm: Family C, the classroom group totals.

"" how does all of these outputs look to you? everything seems like it ran okay

Everything that printed matches exactly, and nothing errored:

Check	Expected	Yours
CSV export	55 of 59 tabs, 20 workbooks	✓
Legacy TOTALS	3,521 checked, 0 mismatches	✓
Legacy point sums > 25	174 of 6,368	✓
Modern Total Points	834 checked, 824 match, 10 mismatch	✓

The 834 rows checked also confirms the modern import kept exactly the right number of quadrats, so the row-exclusion rules worked. (The setup chunk ran twice at the top; that's harmless.)

The one thing I can't see is the tables. `knitr::kable` output goes into the chunk output under each chunk in RStudio, not this console log, so the summary tables (9,074 cells, 104 taxa, 420 excluded, and so on) aren't in what you sent. Rather than screenshotting every table, paste this into the console. It prints PASS or FAIL for every number we haven't confirmed yet:

```
None
chk <- function(label, got, want) cat(sprintf("%-4s %-38s got
%s, want %s\n",

  if (identical(as.numeric(got), as.numeric(want))) "PASS"
else "FAIL", label, got, want))



chk("legacy files",
n_distinct(legacy$surveys$source_file), 9)
```

```
chk("legacy blocks",          nrow(legacy$surveys), 228)
chk("legacy count cells",     nrow(legacy$counts), 17684)
chk("legacy raw taxa",
n_distinct(legacy$counts$taxon_raw), 172)
chk("legacy issues",          nrow(legacy$issues), 2)
chk("modern quadrats",        nrow(modern$quadrats), 834)
chk("modern count cells",     nrow(modern$counts), 9074)
chk("modern raw taxa",
n_distinct(modern$counts$taxon_raw), 104)
chk("modern rows excluded",   nrow(modern$excluded), 420)
chk("modern metadata rows",   nrow(modern$metadata), 39)
chk("modern issues",          nrow(modern$issues), 11)
chk("modern NA categories",
sum(is.na(modern$counts$sheet_category)), 0)
```

If all 12 say PASS, Families A and B are confirmed. If any say FAIL, send me that line. To eyeball the tables themselves, render the report (the Render button) and look at the HTML. That also checks that the whole document renders, which the assignment requires.

The checker could also go in the AI Developer Log: it's the "manual check" of AI output against an independent calculation. Ready to start Family C whenever you confirm.

None

```
 chk <- function(label, got, want) cat(sprintf("%-4s %-38s got
%s, want %s\n",

+                                           if
(identical(as.numeric(got), as.numeric(want))) "PASS" else
"FAIL", label, got, want))

>

> chk("legacy files",
n_distinct(legacy$surveys$source_file), 9)

PASS legacy files                          got 9, want 9

> chk("legacy blocks",          nrow(legacy$surveys), 228)

PASS legacy blocks                         got 228, want 228

> chk("legacy count cells",     nrow(legacy$counts), 17684)

PASS legacy count cells                    got 17684, want
17684

> chk("legacy raw taxa",
n_distinct(legacy$counts$taxon_raw), 172)

PASS legacy raw taxa                       got 172, want 172

> chk("legacy issues",          nrow(legacy$issues), 2)

FAIL legacy issues                         got 229, want 2

> chk("modern quadrats",        nrow(modern$quadrats), 834)

PASS modern quadrats                       got 834, want 834
```

```
> chk("modern count cells",     nrow(modern$counts), 9074)

PASS modern count cells                    got 9074, want
9074

> chk("modern raw taxa",
n_distinct(modern$counts$taxon_raw), 104)

PASS modern raw taxa                       got 104, want 104

> chk("modern rows excluded",   nrow(modern$excluded), 420)

PASS modern rows excluded                  got 420, want 420

> chk("modern metadata rows",   nrow(modern$metadata), 39)

FAIL modern metadata rows                  got 38, want 39

> chk("modern issues",          nrow(modern$issues), 11)

PASS modern issues                         got 11, want 11

> chk("modern NA categories",
sum(is.na(modern$counts$sheet_category)), 0)

PASS modern NA categories                  got 0, want 0
```

None

```
count(legacy$issues, issue)

# A tibble: 2 × 2

  issue                 n

  <chr>             <int>

1 header_inferred       1

2 no_survey_metadata   228

> head(legacy$issues, 3)

# A tibble: 3 × 3

  block_id                     issue        detail

  <chr>                        <chr>        <chr>

1 OPIHI data 2004.xlsx|block01 no_survey_m… date,…

2 OPIHI data 2004.xlsx|block02 no_survey_m… date,…

3 OPIHI data 2004.xlsx|block03 no_survey_m… date,…

> legacy$surveys |> summarise(n = n(), date_missing =
sum(is.na(date)),

+                             site_missing =
sum(is.na(site_raw)), school_missing = sum(is.na(school_raw)))

# A tibble: 1 × 4

      n date_missing site_missing school_missing

  <int>        <int>        <int>          <int>
```

```
1   228          228          228            228

> legacy$surveys |> select(block_id, date_raw, site_raw,
school_raw) |> head(3)

# A tibble: 3 × 4

  block_id            date_raw site_raw school_raw

  <chr>               <chr>    <chr>    <chr>

1 OPIHI data 2004.xl… NA       NA       NA

2 OPIHI data 2004.xl… NA       NA       NA

3 OPIHI data 2004.xl… NA       NA       NA

> count(modern$metadata, source_file)

# A tibble: 4 × 2

  source_file                                    n

  <chr>                                      <int>

1 OPIHI 2024 datasheet .xlsx                    15

2 OPIHI 2026 Longitudinal Monitoring Data.x…    12

3 OPIHI BIOL 499 2025 LT data.xlsx              10

4 RAW Mauliola OPIHI Data 2026.xlsx              1
```

the data in the raw data is the real unredacted data. It is named the same as the data you have, though. are we good to keep moving? I want to get to the final processed data sheets we can use for the rest of the assignment.

Yes, we're good to keep moving. Using the real data in `Data/Raw` is the right call for the final product. I'll just make sure names only ever leave the pipeline as anonymized IDs, and the real-name key gets written to a private file. When you next run, please include the two legacy check lines (`header_inferred` 1 and `no_survey_metadata` 1, plus real dates and sites).

To get you to the final processed sheets fastest, I'll build the remaining steps together: Family C, site and taxon cleaning, Teacher IDs, deduplication, and the master files. Family C first, since it's the last unknown format.

This is the full pipeline, from raw files to final processed data sheets.

Where the files go:

```
None
AI data/

├── Data/

|   ├── Raw/                          (unchanged)

|   ├── Lookups/                      ← NEW folder

|   |   ├── site_crosswalk.csv

|   |   └── taxon_crosswalk.csv

|   └── Processed/                    ← outputs get written here

└── Scripts/

    ├── opihi_report.qmd              ← updated

    ├── helpers.R                     (unchanged)

    ├── import_legacy.R               (the fixed one from
before)

    ├── import_modern.R               (unchanged)

    ├── import_classroom.R            ← NEW
```

```
└── clean_build.R                ← NEW
```

What the pipeline now does beyond the imports:

- **Family C: the classroom group totals, kept as `group_total` records with effort taken from each group's TOTAL Points.**
- **Sites and taxa: cleaned through the two lookup CSVs. 66 site spellings become 44 sites; 300 taxon spellings become 220 clean taxa and 138 analysis taxa. Anything not in a lookup is listed with a suggested closest match.**
- **Site correction by coordinates: 3 legacy trips were labelled with the wrong site; for example, "Barbers Point" had coordinates on Hawai'i Island. Their coordinates match a different site exactly, so they're relabelled and flagged.**
- **Compiled-file cross-check: every trip in the compiled files is checked against the master. No species data exists only in compiled files. The 18 trips that appear only there are the red "no species data" trips, now logged as known gaps.**
- **Teacher IDs (`T###`): the real-name key is written only to `Data/Processed/private/`. Keep that folder out of GitHub.**

Output files in `Data/Processed/`:

File	What it is
`opihi_master.csv`	**The master: one row per detection, with survey, site, teacher ID, sample, taxon, count, and a pointer to the source cell**
`opihi_surveys.csv`	**One row per site visit, with effort (samples, quadrats)**
`opihi_samples.csv`	**One row per quadrat or student group, including empty ones. This is the effort table for detection frequency.**

`opihi_exclusions_log.csv`	Everything left out, and the rule that excluded it
`opihi_qc_flags.csv`	Everything kept but flagged
`opihi_compiled_crosscheck.csv`	The Family D check
`opihi_data_dictionary.csv`	The data dictionary

To check it: run the whole document top to bottom, then paste this into the console:

r

```
None
chk <- function(label, got, want) cat(sprintf("%-4s %-32s got %s, want %s\n",
  if (identical(as.numeric(got), as.numeric(want))) "PASS" else "FAIL", label, got, want))
chk("legacy issues (fixed?)",   nrow(legacy$issues), 2)
chk("classroom groups",         nrow(classroom$groups), 34)
chk("classroom count cells",    nrow(classroom$counts), 440)
chk("classroom issues",         nrow(classroom$issues), 1)
chk("unmatched names",          nrow(opihi$unmatched), 0)
chk("surveys",                  nrow(opihi$surveys), 252)
```

```
chk("samples",                  nrow(opihi$samples), 7216)

chk("master rows",              nrow(opihi$master), 22588)

chk("clean taxa in master",
n_distinct(opihi$master$taxon_clean), 215)

chk("analysis taxa in master",
n_distinct(opihi$master$analysis_taxon), 137)

chk("exclusions",               nrow(opihi$exclusions), 427)

chk("sites fixed by coords",    sum(opihi$qc_flags$flag ==
"site_corrected_by_coordinates"), 3)

chk("compiled-only WITH data",
sum(str_detect(opihi$crosscheck$status, "HAS species")), 0)

count(opihi$exclusions, reason)
```

This is far more code than I could test in R here; I checked its logic line by line and computed these targets independently in Python. If something errors or a line says FAIL, send me the exact message or line and I'll fix it.

For your team to review:

- **The 6 `REVIEW` lines in `taxon_crosswalk.csv`, e.g. *Avrainvillea amadelpha* → *A. lacerata*.**
- **The `K Bay` → Kahana Bay call in the site crosswalk.**

Sent error

Everything ran start to finish without an error, and all seven output files were written to `Data/Processed/`. The data dictionary is there too; it's written after the file listing, which is why it isn't in that list.

One check first: the summary tables (surveys, samples, master rows, exclusions) only show in the chunk output, so I can't see them in what you sent. Please paste the `chk(...)` checker from two messages back and send the result. That confirms the master file has the right contents before you build figures on it.

I also spotted one small display bug: the line `Taxon lookup: … 139 analysis taxa` should say 138. It counts a blank value as a taxon. It's cosmetic (the data aren't affected), and I'll fix it in the next round.

Which files to use for plotting

File	One row per	Use it for
`opihi_master.csv`	detection (taxon recorded in a sample, count > 0)	where and when a taxon was recorded, cover (count / points_possible)
`opihi_samples.csv`	quadrat or student group, including empty ones	sampling effort: the denominator for detection frequency
`opihi_surveys.csv`	site visit	effort per visit (n_quadrats), site, island, date, teacher ID

For Question C, detection frequency for a taxon at a site-visit is:

> quadrats where the taxon was recorded ÷ quadrats sampled

The numerator comes from `opihi_master` and the denominator from `opihi_samples`. Use `resolution == "quadrat"` only, since group totals can't give per-quadrat detection. The join is a few lines, and it's a good candidate for the reusable function.

Assignment requirements: status

Requirement	Rubric pts	Status
1. Inspect & define: observation unit, site, date, taxa, effort fields; question & metric	10	🟡 Mostly done. The fields are identified and documented. Still to do: pick the focal taxon or group and write the question and metric precisely.
2. Clean & document: import raw, convert from Excel, one master, no duplicates, check dates/types/missing values/taxa/sites, explain exclusions, data dictionary	25	✅ Done, pending the checker. Master + surveys + samples, exclusions log, QC flags, crosswalks, data dictionary, compiled-file cross-check. 🟡 Still to do: the "beautiful tidy table" (currently a plain `kable`; I'd switch to `gt`).
3. Reusable function: inputs/output explained, tested, useful to OPIHI for new data	part of 25	❌ Not yet. Many functions exist, but none is presented and tested as *the* function.
4. Two figures with captions, exploratory	15	❌ Not yet
5. Reproducible: GitHub repo, relative paths, README, metadata, Quarto report renders, published HTML	part of 25	🟡 Relative `here()` paths, `.Rproj`, package check, session info, and data dictionary are done. Still to do: README, `.gitignore`, a full render, the repo, and publishing.
6. AI Developer Log: tools, prompts, outputs, a concrete AI mistake,	15	🟡 Plenty of material, not written up.

debugging, manual checks, disclosure		
Ethics, collaboration, individual reflections	**10**	**🟡 Teacher IDs plus a private key, student names kept out, real data kept off GitHub. Reflections are individual.**

What remains, in order

1. **Run the checker and confirm the numbers.**

2. **Choose the focal taxon or group. Everything is clean, so this is purely a scientific choice. Options:**

    - **invasive algae (*Acanthophora*, *Avrainvillea*, *Gracilaria salicornia*)**
    - ***Turbinaria ornata***
    - **urchins (*Echinometra*, *Tripneustes*, *Colobocentrotus*)**
    - **‘opihi (*Cellana*). It's culturally fitting, but it was only detected about 12 times, which is worth knowing before you commit.**
3. **The reusable function, with tests. Two good options:**

    - **A detection-frequency calculator: give it a taxon, get detection frequency per site-visit with effort. It powers both figures.**
    - **A new-datasheet checker: flags unknown sites and taxa with suggested matches, point sums over 25, missing dates, and duplicates. This matches the assignment's "check for errors in new incoming data" wording. We already have most of the pieces.**
4. **One function is enough. I'd do the checker as *the* tested function and use the calculator as a helper.**

5. **Two figures. For example:**

    - **a site × year detection map or heatmap, showing where and when, with effort**
    - **detection frequency over time at repeatedly sampled sites**
6. **Tidy table with `gt`.**

7. **README + `.gitignore`. `Data/Raw/` (real names) and `Data/Processed/private/` must stay out of the public repo. The README explains how someone with permitted data reruns everything.**

8. **AI Developer Log. It'll draw on what we've collected:**

    - **the hidden 2017 trip merged into its neighbor**
    - **the metadata off-by-one, caught by the checker**
    - **the chart sheets that crashed `readxl`**
    - **`coalesce()` on a matrix**
    - **the 1 km site threshold that would have mislabelled real sites**
    - **Python vs R verification throughout**

9. **Render, push, publish the HTML.**

Send the checker output and your focal taxon choice, and I'll do steps 3–5 next.

Let's get #1 taken care of, then give instructions for how we can proceed with 2-6. I already have 7 done. do not do the instructions for 2-6 just yet, we need to take care of 1 first, but once we get there please explain which sheets will be most useful from processed data and why

Understood, I won't bring that up again. I also removed the one place the name appeared in the code: a comment in the output chunk. The updated `opihi_report.qmd` also fixes the "139 analysis taxa" count line, which should now print 138.

Step 1: confirm the final numbers. Replace the report in `Scripts` and re-run it top to bottom. Then paste this into the console and send me everything it prints:

None

```
chk <- function(label, got, want) cat(sprintf("%-4s %-32s got
%s, want %s\n",

  if (identical(as.numeric(got), as.numeric(want))) "PASS"
else "FAIL", label, got, want))

chk("legacy issues",            nrow(legacy$issues), 2)

chk("classroom groups",         nrow(classroom$groups), 34)

chk("classroom count cells",    nrow(classroom$counts), 440)

chk("classroom issues",         nrow(classroom$issues), 1)

chk("unmatched names",          nrow(opihi$unmatched), 0)

chk("surveys",                  nrow(opihi$surveys), 252)

chk("samples",                  nrow(opihi$samples), 7216)

chk("master rows",              nrow(opihi$master), 22588)

chk("clean taxa in master",
n_distinct(opihi$master$taxon_clean), 215)

chk("analysis taxa in master",
n_distinct(opihi$master$analysis_taxon), 137)

chk("exclusions",               nrow(opihi$exclusions), 427)

chk("sites fixed by coords",    sum(opihi$qc_flags$flag ==
"site_corrected_by_coordinates"), 3)
```

```
chk("compiled-only WITH data",
sum(str_detect(opihi$crosscheck$status, "HAS species")), 0)

count(opihi$exclusions, reason)

count(opihi$crosscheck, status)
```

The last two lines aren't pass/fail. They show the breakdown, so if a total is off, I can see exactly where. If every line says PASS, the processed data are confirmed. Then I'll explain which processed sheets are most useful for the rest of the assignment and why, and lay out how to proceed with steps 2–6.

Okay so now that all of the data is cleaned (correct?) let's lay out what we want to do for steps 2-6. can you describe what was done, what all of the cleaned outputs are, what will be most useful, what tasks remain. Please be clear. Soome questions we have, or things that we want to note so that others can remember are... Can we pull out specific locations from the data and which sheets would be most useful for this because we want to make a figure with the locations. What is in each file that was created? what are the various aspects of the codes doing? Just good descriptions on what was done and what the outputs are, column names ,etc, because the people using this information tomorrow and in the future will be relatively unfamiliar with these steps and outputs. we want to be as clear as possible. thanks!

Yes, the data are cleaned and verified. All 13 checks passed against an independent calculation. Since this is a reference your team will keep coming back to, I'll write it up as a shareable document rather than a long chat reply.

Outline is up; now filling the status section.

Filling the status section first.

Now the folder layout and how to rerun.

Now the raw data families.

Now what the code does, file by file.

Now the cleaning rules.

Now every output file and its columns, the longest section.

Before writing the map section, verifying that every site in the master can get coordinates (only 2004–2020 datasheets record them):

All 43 sites that appear in the master have coordinates from at least one 2004–2020 visit, so a site-level map is fully possible. Writing that section.

Now how to calculate detection frequency correctly.

Now the caveats.

Last section: remaining tasks and the decisions they need.

All sections are filled. One last addition: section 4 describes a flow, which reads faster as a picture for someone new to the project, so I'm adding a pipeline diagram under that section's lead.

Checking how the diagram renders:

The diagram renders cleanly. Leaving one question in the doc for your team:

The handoff guide is ready: share it with your team, edit inline, or leave comments and I'll revise.

TABLE CREATION

Using our cleaned OPIHI outputs (Data/Processed/opihi_master.csv, opihi_samples.csv,

opihi_surveys.csv) and our team's algae origin lookup (Data/Algae/algae_origin_lookup.csv,

with any corrections in algae_origin_review.csv's your_status column), write a standalone

Quarto document, Scripts/opihi_algae_table.qmd, that builds one publication-ready gt table

comparing invasive and native algae in OPIHI surveys, 2004–2026.

Requirements:

- Use quadrat-level records only; exclude classroom group totals and cyanobacteria, and say so.

- Account for sampling effort: report % of quadrats where each taxon was recorded, with the

denominator taken from opihi_samples (all quadrats, including empty ones), split into the

three sampling periods (2004–2008, 2016–2020, 2024–2026) plus all years. Put the number of

sites and quadrats sampled in each column header.

- Distribution: number of sites and islands where recorded. Abundance: mean % cover where

present (points / 25).

- Rows: a summary block for each origin status, every invasive species, and the 8 most

widespread native species; write all taxa to Outputs/algae_origin_summary.csv.

- Italicize scientific names, add footnotes for caveats (uneven sites between periods,

cover comparability, provisional native status) and cite sources for invasive status.

- Use here() paths, a package check, no teacher IDs, and no statistics (exploratory only).

- Save the table to Outputs/ as a PNG and an HTML file.

- Start the document with a short table explaining each design choice and which assignment

requirement it meets.

our group already classified every alga in `algae_origin_review.csv`, so the table will use **your** origin statuses rather than mine. That keeps the table consistent with your teammates' figures. Recomputing the numbers with your classification:

Two fixes before sending: passing `...` into `join_by()` is unreliable, and the "Reading the table" bullets should pull their numbers from the data instead of having them typed in.

I've written `opihi_algae_table.qmd`, but I couldn't run R here, so it hasn't been run yet. I checked the numbers in Python against the cleaned data instead; the check values are below.

It's a standalone Quarto document: it reads the cleaned files and builds one `gt` table comparing invasive and native algae.

Where things go:

- Save the document as `Scripts/opihi_algae_table.qmd`.
- It reads `Data/Processed/` and `Data/Algae/algae_origin_lookup.csv`. It also reads `algae_origin_review.csv` from the same folder, if it's there.
- It writes to `Outputs/`:
    - `algae_invasive_native_table.png`
    - `algae_invasive_native_table.html`
    - `algae_origin_summary.csv`, with every algal taxon, not just those shown.

What the table shows:

- **Rows:** one block per origin status (from your team's lookup), then the 4 invasive species, then the 8 most widespread native species.
- **When:** % of quadrats where each alga was recorded, for 2004–2008, 2016–2020, 2024–2026 and all years. Each column header shows how many sites and quadrats were sampled.
- **Where:** number of sites and islands.
- **Abundance:** mean % cover where present.

Footnotes cover the sources for invasive status and the main caveat: 2024–2026 comes from only 4 O‘ahu sites. The document opens with a short table explaining each design choice and the assignment requirement it meets.

Two things to know:

- **The PNG needs Google Chrome installed**, because `gt` uses it through the `webshot2` package. If saving the PNG fails, the HTML copy still saves and the message tells you why.
- **Your lookup marks *Turbinaria ornata* as "Unresolved (check)".** As far as I know it's native to Hawai‘i; it's invasive elsewhere in the Pacific. I left your team's call as is. If you fill `Native (likely)` into `your_status` in the review CSV, the table picks it up automatically.

Check values for after you run it:

Row	2004–08	2016–20	2024–26	All years	Sites	Islands	Cover
Effort (quadrats / sites)	1,356 / 13	4,992 / 42	834 / 4	7,182 / 43			
Invasive species	24.1	12.3	18.6	15.2	24	6	19.0
Native species	24.9	18.5	41.4	22.3	41	6	18.1
Acanthophora spicifera	20.0	7.8	4.3	9.7	22	6	20.1
Identified to genus only	62.0	49.8	85.1	56.2	41	6	37.2

Your data are unredacted, so the last decimal place could differ slightly. Send me the PNG once it renders and I'll check the layout.

Jasmine’s AI Log

AI Conversations

Differentiating native algae from invasive algae. When asking AI (Claude) to clean the master sheet and create new sheets only displaying algae data, I mentioned that I planned to differentiate native and non-native limu. Claude did this on it‘s own, with it‘s own sources and created a new data sheet with the algae species organized as either "Native", "Non-native(Invasive)", or "Unresolved." As seen in conversation and data below:

****ME**: I can first make a new data sheet that only has algae. Then I can plot and interactive map that shows where the native vs non-native algae was found around the island. Secondly, I want to make a plot showing the abundance of native vs non-native species on a time series line graph for each location. So it would be two lines and I would change out the location for each, resulting in around 8 graphs?**

****CLAUDE**: I wrote algae_map_and_timeseries.R, which covers all three steps. I haven’t run it, since R isn’t available in my sandbox, so expect to fix a small thing or two on the first run.**

What the script does

1. Algae-only sheet. It writes algae_only.csv, with each record labeled Native, Non-native, or Unresolved. It also writes a companion quadrat_samples.csv listing every quadrat surveyed, including ones with no algae. Without that second file, empty quadrats disappear from an algae-only sheet and average cover comes out too high.

2. Interactive map. Each location gets a green circle for native algae and a red circle for non-native algae, sized by mean % cover. The circles are offset a few hundred meters east and west so both stay visible, and you can toggle each layer. Clicking a site shows its cover values, number of quadrats, and survey years. The map is saved as oahu_native_vs_nonnative_map.html.

3. Time series. There are two lines per location, native and non-native. plot_site("Leahi") draws one site, and changing the name switches locations. The script also saves a PNG for each site and one overview page with all of them.

I made a review sheet and a script that merges your decisions back, so you don't have to edit the lookup by hand. I haven't run the R script.

How to clean it up

1. Open algae_origin_review.csv. It lists 109 taxa, with the ones that most need a decision first. The first 30 rows are the "check" taxa, such as Turbinaria ornata, Spyridia filamentosa, and Colpomenia sinuosa, followed by the 46 genus-level entries, then my 29 "likely native" and 4 non-native calls to confirm.

2. Fill in your_status using only Native, Non-native, Cryptogenic, or Unresolved. Put your source in source_or_note. Leave a row blank to keep the current status.

3. Run update_origin_lookup.R. It checks that your entries are valid, writes an updated algae_origin_lookup.csv, keeps a backup of the old one, and prints what's still undecided.

Re-run algae_map_and_timeseries.R. I updated it to accept the new labels, and the map and graphs will pick up your changes. Cryptogenic and Unresolved taxa are left off the native and non-native lines.

4. Where to look things up

*** UH Reefalgae field guide (hawaii.edu/reefalgae), which marks algae as alien, native, or unknown origin.**

*** AlgaeBase, filtered to the Hawaiian Islands.**

*** Carlton & Eldredge (2009), the Bishop Museum list of introduced and cryptogenic marine species in Hawai‘i.**

*** A local expert. Your organization may already have someone who can confirm the less certain taxa.**

All of the algae from these data sheets were cross checked with the book ‘Hawaiian Reef Plants‘ as well as Algae Base.

Carolina's AI Log

Fixing the final script

Had a duplicate chunk label

The error is:

Error in `tab_options()`:

! could not find function "tab_options"

This happens because tab_options() belongs to the gt package, but your script isn't loading that package.

Step 1: Fix your package-loading section

In your ## First Steps section, you currently have:

#library(gt)

Remove the # so it becomes:

library(gt)

If you haven't installed gt, run this once in your R Console:

install.packages("gt")

Step 2: Save and render again

Save your .qmd file and click Render.

Alternative: Make the function explicit

Instead of loading the whole package, you can change:

tab_options(

to:

gt::tab_options(

Fixing Duplicate chunks

Step 1: Find the duplicate chunks

In RStudio, open FinalScript_GroupC.qmd.

Press Command + F (Mac) and search for:

```{r setup}

You should find this label in two different places in your document.

You might also have a chunk written like this:

```{r}

#| label: setup

```

Step 2: Rename one of them

Since your first setup chunk loads all your packages, I would keep that one named setup.

For the second chunk, change its name to something else.

Before:

```{r setup}

Your code here

```

After:

```{r setup-2}

Your code here

```

Trying to render issues

What I sent: rocessing file: FinalScript_GroupC.qmd

|..... | 73% [flag-details-check] [flag-details]

Error in xlsx_cells():

! could not find function "xlsx_cells"

Backtrace:

■

1. └─dplyr::filter(...)

Quitting from FinalScript_GroupC.qmd:1458-1483 [flag-details-check]

Execution halted

WARN: Error encountered when rendering files

What AI told me to do:

Step 1: Find the function

In FinalScript_GroupC.qmd, press Command + F and search for:

xlsx_cells(

Step 2: Make one small change

Original:

xlsx_cells(

Corrected:

tidyxl::xlsx_cells(

Step 3: Save and render again

You can also search the entire document for any remaining xlsx_cells(calls and make the same correction.

Why this happens: xlsx_cells() belongs to the tidyxl package. Using tidyxl::xlsx_cells() tells R exactly where to find it, even when the package hasn't been loaded with library(tidyxl)

Creating functions

unction 1: Taxon richness at each site

Your goal: Create a function that counts how many different taxa were recorded at each sampling site.

What AI helped create:

```
TaxonRichness <- function(data) {


  result <- data |>

    filter(!is.na(site_name)) |>

    group_by(site_name) |>

    summarise(

      richness = n_distinct(analysis_taxon, na.rm = TRUE),

      .groups = "drop"

    )


  return(result)

}
```

Function 2: Taxon richness at each site by year

Your goal: Modify the first function to calculate richness separately for each site and sampling year.

What we changed: Added year to the grouping and arranged the results by site and year.

```
TaxonRichness <- function(data) {


  result <- data |>

    filter(!is.na(site_name), !is.na(year)) |>

    group_by(site_name, year) |>
```

```
    summarise(
      richness = n_distinct(analysis_taxon, na.rm = TRUE),
      .groups = "drop"
    ) |>
    arrange(site_name, year)


  return(result)
}
```

Linnea's AI Log

See new updated PDF